

Name of Student	<u>Shreyash Ganesh Dhekane</u>
Class Roll No	<u>16</u>
D.O.P.	<u>08-04-25</u>
D.O.S.	<u>15-04-25</u>
Sign and Grade	

EXPERIMENT NO. 9

AIM: To study AJAX

Theory:

1. How do Synchronous and Asynchronous Requests differ?

Synchronous vs. Asynchronous Requests

The difference between synchronous and asynchronous requests in the context of web development lies in how they handle tasks and interactions during the communication between the client and server.

Synchronous Requests:

A synchronous request means that the JavaScript execution will pause and wait for the server's response before moving on to the next task. The code execution is blocked until the response is received, meaning no other actions can take place during the wait. This can cause a delay or freeze in the user interface, especially if the server response takes time. Synchronous requests are not commonly used in modern web development because they can negatively impact user experience.

Example:

```
var xhr = new XMLHttpRequest();
xhr.open("GET", "example.txt", false); // 'false' makes the request synchronous
xhr.send();
console.log(xhr.responseText); // This will run only after the request completes
```

Asynchronous Requests:

An asynchronous request allows JavaScript to send a request to the server and then continue executing without waiting for the server's response. The server response is handled in the background, and once the data is received, a callback function or promise is used to process the response. This is the preferred method in modern web development because it ensures that the user interface remains responsive while waiting for the server's response.

Example:

```
var xhr = new XMLHttpRequest();
xhr.open("GET", "example.txt", true); // 'true' makes the request asynchronous
xhr.onload = function() {
    console.log(xhr.responseText); // Process the response once it's available
};
xhr.send();
console.log("Request sent"); // This will run immediately after the request is sent
```

2. Describe various properties and methods used in XMLHttpRequest Object

XMLHttpRequest Object: Properties and Methods

The XMLHttpRequest (XHR) object is used in JavaScript to interact with servers and handle HTTP requests and responses. It is essential for making AJAX (Asynchronous JavaScript and XML) calls.

Common Properties of XMLHttpRequest:

- **readyState**: Indicates the current state of the request.
 - 0: UNSENT – The request has not yet been opened.
 - 1: OPENED – The request has been opened but not sent.
 - 2: HEADERS_RECEIVED – The request has been sent, and the response headers are received.
 - 3: LOADING – The response body is being received.
 - 4: DONE – The request is complete, and the response is fully received.
- **status**: The HTTP status code of the response (e.g., 200 for success, 404 for not found).
- **statusText**: A textual description corresponding to the status code (e.g., "OK" for a successful request).
- **responseText**: The response data as a string (used when the response is textual, such as HTML or JSON).
- **responseXML**: The response data as an XML document (used when the response is XML).
- **timeout**: The number of milliseconds before the request times out. A timeout triggers an ontimeout event.

Common Methods of XMLHttpRequest:

- **open(method, url, async, user, password)**:
- **method**: The HTTP method (e.g., "GET", "POST").
- **url**: The URL to send the request to.
- **async**: A boolean indicating if the request should be asynchronous (true or false).
- **user and password**: Optional parameters for HTTP authentication.

Example:

```
xhr.open("GET", "example.txt", true);
```

send(data): Sends the HTTP request to the server. If the method is POST, the data parameter can include the body of the request (e.g., form data, JSON).

Example:

```
xhr.send();
```

setRequestHeader(header, value): Sets the value of a specified HTTP request header. This is used before sending the request, especially for POST requests.

Example:

```
xhr.setRequestHeader("Content-Type", "application/json");
```

abort(): Cancels the current request, if it is still in progress.

Example:

```
xhr.abort();
```

getResponseHeader(header): Returns the value of a specified response header from the server.

Example:

```
var contentType = xhr.getResponseHeader("Content-Type");
```

getAllResponseHeaders(): Returns all the response headers as a string.

Example:

```
var allHeaders = xhr.getAllResponseHeaders();
```

onreadystatechange: An event handler that is called whenever the readyState property changes.

Example:

```
xhr.onreadystatechange = function() {  
    if (xhr.readyState == 4 && xhr.status == 200) {  
        console.log(xhr.responseText);  
    }  
};
```

onload: An event handler that is triggered when the request is completed successfully.

Example:

```
xhr.onload = function() {  
    console.log(xhr.responseText);  
};
```

onerror: An event handler that is triggered when the request fails.

Example:

```
xhr.onerror = function() {  
    console.error("Request failed");  
};
```

In modern web development, it is common to use the Fetch API as a replacement for XMLHttpRequest due to its simpler syntax and support for promises. However, XMLHttpRequest is still widely used and important for understanding how AJAX works at a fundamental level.

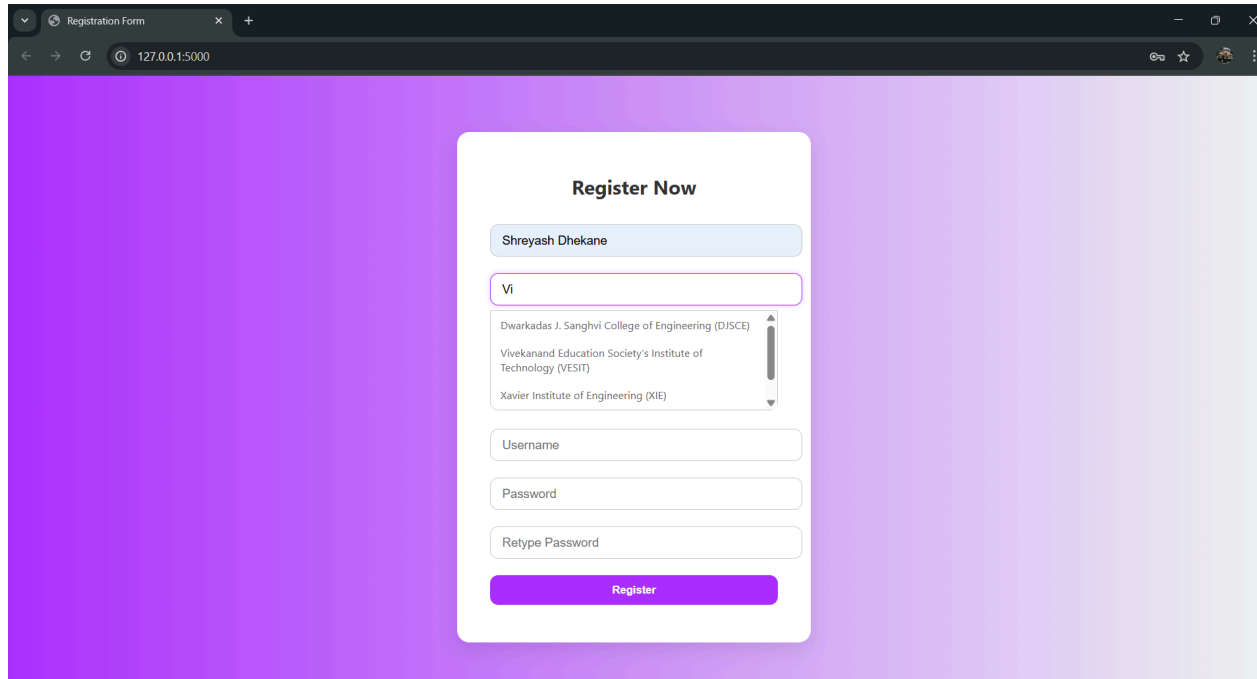
TASK PERFORMED:

In this task, I created a registration page with fields for Name, College, Username, and Password (with a confirmation field to retype the password). The form was designed to validate user input by ensuring that the username is unique and not already in use, the Name field is not left empty, and the retyped password matches the original one.

To improve user experience, I implemented an auto-suggest feature for the college name field, providing users with suggestions as they type. Additionally, I used JavaScript's XMLHttpRequest object to handle asynchronous updates, allowing the form to submit without refreshing the page. Upon successful registration, a message saying "Successfully Registered" is displayed below the submit button, confirming that the user has been registered successfully. This entire process was handled asynchronously to enhance the user experience and ensure smooth interaction with the form.

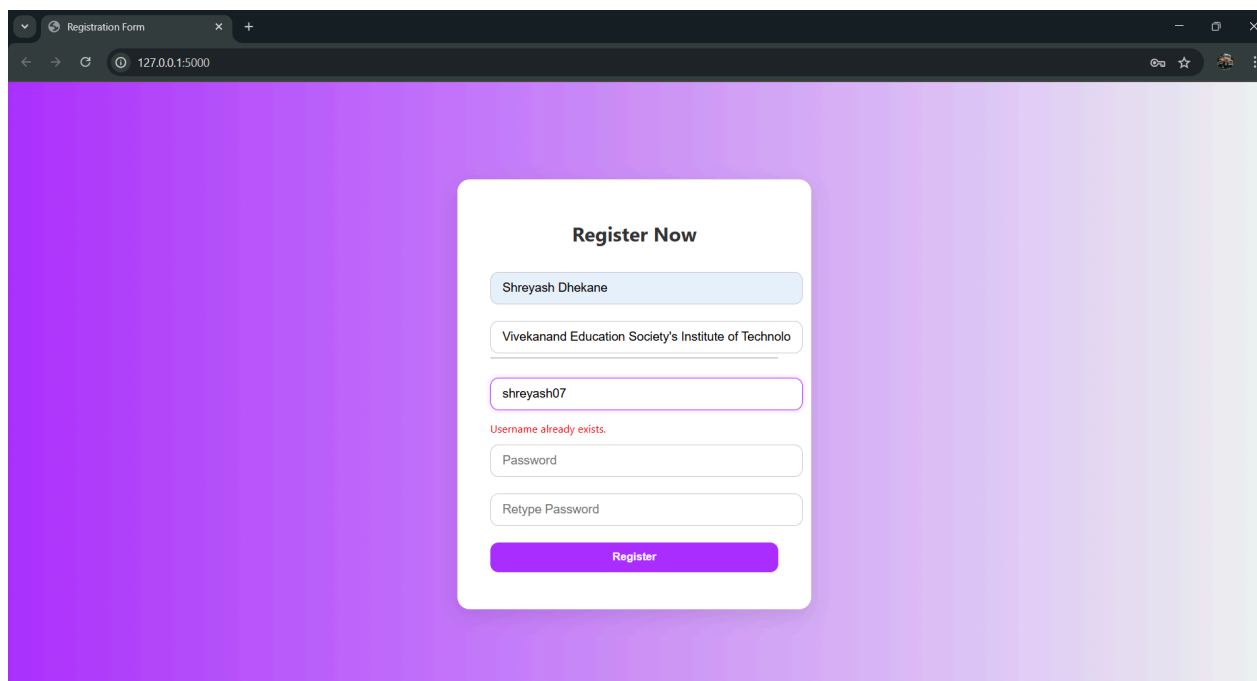
OUTPUT: -

- Demonstration of dynamic college name suggestions as the user types into the College field.



A screenshot of a web browser displaying a registration form titled "Register Now". The form is centered on a purple gradient background. It contains several input fields: a name field with "Shreyash Dhekane", a college field with "Vi" and a dropdown menu showing suggestions like "Dwarkadas J. Sanghvi College of Engineering (DJSC)", "Vivekanand Education Society's Institute of Technology (VESIT)", and "Xavier Institute of Engineering (XIE)", and fields for "Username", "Password", and "Retype Password". A red "Register" button is at the bottom.

- Validation alert shown when the entered username already exists in the database.



A screenshot of the same "Register Now" form, but with a validation alert. The "Username" field now contains "shreyash07", and a red error message "Username already exists." is displayed below it. The "College" field now shows "Vivekanand Education Society's Institute of Technolo". The "Register" button remains at the bottom.

Registration Form

127.0.0.1:5000

Register Now

Shreyash Dhekane

Vivekanand Education Society's Institute of Technolo

shreyashdhekane07

Username is available.

Password

Retype Password

Register

- Message displayed when Password and Retype Password fields do not match.

Registration Form

127.0.0.1:5000

Register Now

Shreyash Dhekane

Vivekanand Education Society's Institute of Technolo

shreyashdhekane07

Username is available.

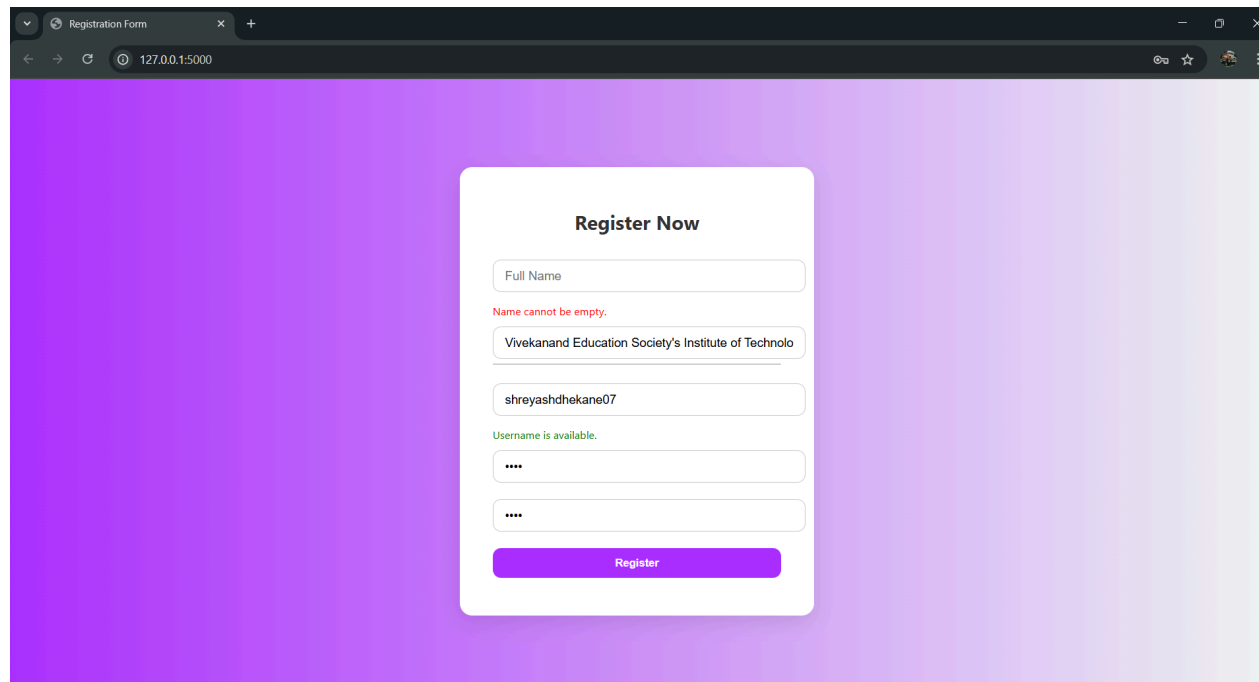
....

....

Passwords do not match.

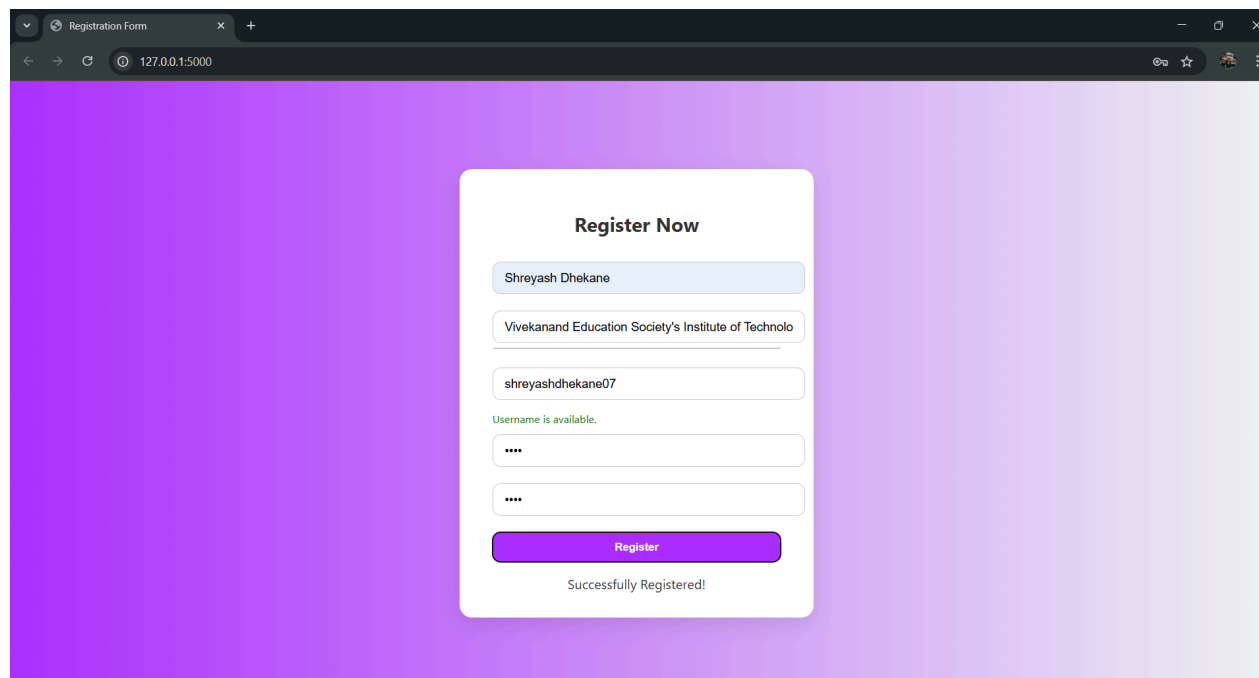
Register

- Error message displayed when the Name field is left empty.



The screenshot shows a web browser window with the title "Registration Form" and the address bar displaying "127.0.0.1:5000". The page has a purple gradient background. In the center, there is a white rounded rectangle titled "Register Now". Inside this rectangle, there are four input fields: "Full Name", "Vivekanand Education Society's Institute of Technolo", "shreyashdhekane07", and two password fields represented by asterisks. A red error message "Name cannot be empty." is displayed below the "Full Name" field. A green message "Username is available." is displayed below the username field. At the bottom of the white rectangle is a purple "Register" button.

- "Successfully Registered!" message displayed after correct form submission.



The screenshot shows the same web browser window as before, but now the "Full Name" field is filled with "Shreyash Dhekane" and has a light blue background. The red error message is gone. The "Register" button is still present. Below the "Register" button, the text "Successfully Registered!" is displayed in a small font.

Conclusion: -

In this experiment, we developed a responsive registration form using **AJAX** on the frontend and **Flask** as the backend, enabling asynchronous communication between the client and the server. Using the XMLHttpRequest object, we created a registration form that validates inputs such as username availability, empty name fields, and password matching in real-time. We also added an auto-suggestion feature for college names based on user input. All updates were

handled without refreshing the page, showcasing the responsiveness of AJAX. This practical helped us understand how AJAX improves interactivity and performance in modern web applications.